

分治法复杂度

回溯法

0-1 背包问题时间复杂度 2^n

子集树 计算上界 $O(n)$ 最坏情况 $O(2^N)$ 个右儿子计算上界 复杂度 $O(n2^n)$

TSP 问题

N 皇后问题

图的 m 着色问题

检查扩展结点每个儿子所对应的颜色 $O(mn)$, 复杂度 $O(nm^n)$

最大团问题

复杂度 $O(n2^n)$

1、n 后问题回溯算法

(1) 用二维数组 $A[N][N]$ 存储皇后位置, 若第 i 行第 j 列放有皇后, 则 $A[i][j]$ 为非 0 值, 否则值为 0。

(2) 分别用一维数组 $M[N]$ 、 $L[2*N-1]$ 、 $R[2*N-1]$ 表示竖列、左斜线、右斜线是否放有棋子, 有则值为 1, 否则值为 0。

for($j=0; j<N; j++$)

```
    if( 1 ) /*安全检查*/
    {  $A[i][j]=i+1$ ; /*放皇后*/
      2;
      if( $i==N-1$ ) 输出结果;
      else 3; ; /*试探下一行*/
      4; /*去皇后*/
      5; ;
    }
```

1、n 后问题回溯算法

(1) $!M[j] \&\& !L[i+j] \&\& !R[i-j+N]$

(2) $M[j]=L[i+j]=R[i-j+N]=1$;

(3) try($i+1, M, L, R, A$)

(4) $A[i][j]=0$

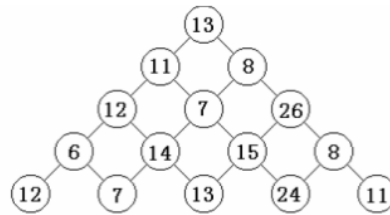
(5) $M[j]=L[i+j]=R[i-j+N]=0$

分别为 $i+j$ 和 $i-j+n$

2、数塔问题。有形如下图所示的数塔，从顶部出发，在每一结点可以选择向左走或是向右走，一起走到底层，要求找出一条路径，使路径上的值最大。

for (r=n-2; r>=0; r--) //自底向上递归计算

```
for (c=0; _____ 1 _____; c++)
    if ( t[r+1][c]>t[r+1][c+1]) _____ 2 _____;
        else _____ 3 _____ ;
```



3、Hanoi 算法

Hanoi (n, a, b, c)

2、数塔问题。

(1) $c \leq r$

(2) $t[r][c] += t[r+1][c]$

(3) $t[r][c] += t[r+1][c+1]$

4、Dijkstra 算法求单源最短路径

$d[u]$: s 到 u 的距离 $p[u]$: 记录前一节点信息

Init-single-source(G, s)

for each vertex $v \in V[G]$

do { $d[v] = \infty$; _____ 1 _____ }

$d[s] = 0$

Relax(u, v, w)

if $d[v] > d[u] + w(u, v)$

then { $d[v] = d[u] + w[u, v]$;

_____ 2 _____
}

dijkstra(G, w, s)

1. Init-single-source (G, s)

2. $S = \Phi$

3. $Q = V[G]$

4. while $Q \neq \Phi$

do $u = \min(Q)$

$S = S \cup \{u\}$

for each vertex _____ 3 _____

do _____ 4 _____

4、(1) $p[v] = \text{NIL}$

(2) $p[v] = u$

(3) $v \in \text{adj}[u]$

(4) Relax(u, v, w)

输出：n 个矩阵链乘所需的数量乘法的最少次数。

for i=1 to n C[i, i]=_____ (1)

for d=1 to n-1

for i=1 to n-d

j=_____ (2)

C[i, j]= ∞

for k=i+1 to j

x= _____ (3)

if $x < C[i, j]$ then

_____ (4) =x

end if

end for

end for

end for

return _____ (5)

end MATCHAIN

2. (1) 0 (2) i+d (3) $C[i, k-1] + C[k, j] + r[i] * r[k] * r[j+1]$

(4) C[i, j] (5) C[1, n]

3. (14 分) 下面是用回溯法求解马的周游问题的算法。

马的周游问题：给出一个 $n \times n$ 棋盘，已知一个中国象棋马在棋盘上的某个起点位置 (x_0, y_0) ，求一条访问每个棋盘格点恰好一次，最后回到起点的周游路线。（设马走日字。）

算法 HORSETRAVEL

输入：正整数 n ，马的起点位置 (x_0, y_0) ， $1 \leq x_0, y_0 \leq n$ 。

输出：一条从起点始访问 $n \times n$ 棋盘每个格点恰好一次，最后回到起点的周游路线；若问题无解，则输出 no solution。

```
tag[1..n, 1..n]=0
dx[1..8]={2, 1, -1, -2, -2, -1, 1, 2}      dy[1..8]={1, 2, 2, 1, -1, -2,
-2, -1}

flag=false
x=x0; y=y0 ; tag[x, y]=1
m=n*n
i=1; k[i]=0
while ____ (1) ____ and not flag
    while k[i]<8 and not flag
        k[i]= ____ (2) ____
        x1= x+dx[k[i]]; y1= y+dy[k[i]]
        if ((x1,y1)无越界 and tag[x1, y1]=0) or ((x1,y1)=(x0,y0) and i=m)
            then
                x=x1; y=y1
                tag[x, y]= ____ (3) ____
                if i=m then flag=true
            else
                i= ____ (4) ____
                ____ (5) ____
```

```

        end if
    end if
end while
i=i-1
_____(6)____
_____(7)____
end while
if flag then outputroute(k) //输出路径
else output “no solution”
end HORSETRAVEL

```

3. (1) $i \geq 1$ (2) $k[i]+1$ (3) 1
 (4) $i+1$ (5) $k[i]=0$ (6) $\text{tag}[x, y]=0$
 (7) $x=x-dx[k[i]]$; $y=y-dy[k[i]]$

n

四. 算法设计题（15 分）

- 一个旅行者要驾车从 A 地到 B 地，A、B 两地间距离为 s 。A、B 两地之间有 n 个加油站，已知第 i 个加油站离起点 A 的距离为 d_i 公里， $0=d_1 < d_2 < \dots < d_n \leq s$ ，车加满油后可行驶 m 公里，出发之前汽车油箱为空。应如何加油使得从 A 地到 B 地沿途加油次数最少？给出用贪心法求解该最优化问题的贪心选择策略，写出求该最优化问题的最优值和最优解的贪心算法，并分析算法的时间复杂性。

四. 算法设计题：

- 贪心选择策略：从起点的加油站起每次加满油后不加油行驶尽可能远，直至油箱中的油耗尽前所能到达的最远的油站为止，在该油站再加满油。

算法 MINSTOPS

输入：A、B 两地间的距离 s ，A、B 两地间的加油站数 n ，车加满油后可行驶的公里数 m ，存储各加油站离起点 A 的距离的数组 $d[1..n]$ 。

输出：从 A 地到 B 地的最少加油次数 k 以及最优解 $x[1..k]$ ($x[i]$ 表示第 i 次加油的加油站序号)，若问题无解，则输出 no solution。

$d[n+1]=s$; //设置虚拟加油站第 $n+1$ 站。

for $i=1$ to n

if $d[i+1]-d[i]>m$ then

```

        output "no solution"; return //无解，返回
    end if
end for
k=1; x[k]=1 //在第 1 站加满油。
s1=m //s1 为用汽车的当前油量可行驶至的地点与 A 点的距离
i=2
while s1<s
    if d[i+1]>s1 then //以汽车的当前油量无法到达第 i+1 站。
        k=k+1; x[k]=i //在第 i 站加满油。
        s1=d[i]+m //刷新 s1 的值
    end if
    i=i+1
end while
output k, x[1..k]
MINSTOPS

```

最坏情况下的时间复杂性： $\Theta(n)$

7. 最长公共子序列问题：给定 2 个序列 $X=\{x_1, x_2, \dots, x_m\}$ 和 $Y=\{y_1, y_2, \dots, y_n\}$ ，找出 X 和 Y 的最长公共子序列。

由最长公共子序列问题的最优子结构性质建立子问题最优值的递归关系。用 $c[i][j]$ 记录序列 X_i 和 Y_j 的最长公共子序列的长度。其中， $X_i=\{x_1, x_2, \dots, x_i\}$ ； $Y_j=\{y_1, y_2, \dots, y_j\}$ 。当 $i=0$ 或 $j=0$ 时，空序列是 X_i 和 Y_j 的最长公共子序列。故此时 $C[i][j]=0$ 。其它情况下，由最优子结构性质可建立

$$\text{递归关系如下: } c[i][j]=\begin{cases} 0 & i=0, j=0 \\ c[i-1][j-1]+1 & i, j > 0; x_i = y_j \\ \max\{c[i][j-1], c[i-1][j]\} & i, j > 0; x_i \neq y_j \end{cases}$$

在程序中， $b[i][j]$ 记录 $C[i][j]$ 的值是由哪一个子问题的解得到的。

(1) 请填写程序中的空格，以使函数 LCSLength 完成计算最优值的功能。

```
void LCSLength(int m, int n, char *x, char *y, int **c, int **b)
{
    int i, j;
    for (i = 1; i <= m; i++) c[i][0] = 0;
    for (i = 1; i <= n; i++) c[0][i] = 0;
    for (i = 1; i <= m; i++)
        for (j = 1; j <= n; j++) {
            if (x[i]==y[j]) {
                c[i][j]=c[i-1][j-1]+1; b[i][j]=1;}
            else if (c[i-1][j]>c[i][j-1]) {
                c[i][j]=c[i-1][j]; b[i][j]=2;}
        }
}
```